

**Document number:** P3212R0  
**Date:** 2024-07-03  
**Audience:** SG21, EWG, LEWG  
**Reply-to:** Andrzej Krzemiński <akrzemi1 at gmail dot com>

# The contract of `sort()`

This paper lists all parts of the contract of function template `std::sort` and demonstrates which parts thereof can be reflected with *contract assertions* proposed in [\[P2900R7\]](#) (Contracts for C++) or with the envisioned their future additions. This is an exercise to test how much of the Standard Library specification can be mapped onto contract assertions.

## 1. The *plain language* contract of `sort()`

We only select one function template declaration for analysis

```
template<random_access_iterator I,  
        sentinel_for<I> S,  
        class Comp = ranges::less,  
        class Proj = identity>  
requires sortable<I, Comp, Proj>  
constexpr I  
    ranges::sort(I first, S last, Comp comp = {}, Proj proj = {});
```

This function has *plenty* of requirements that it imposes on the callers, of different kind, not counting the type-system (syntactic) ones.

First, it uses concepts. Concepts have both syntactic and semantic requirements. While syntactic requirements are type-system-checked by the compiler, the responsibility of satisfying the semantic requirements falls on the caller. There is a difference between *satisfying* syntactic requirements of a concept and *modeling* a concept.

For instance the passed iterator is required (among other things) to be *movable*. But how one can runtime-test if the move constructor actually moves? The only way to test it would be to perform a copy.

Second, there are more complex concepts involved:

- `regular_invocable`
- `strict_weak_order`
- `permutable`

This requires that function `comp` be *equality-preserving* over the elements in range `[first, last)`. Thus, it is ok if `comp` does not guarantee the strict weak order for some values `x` and `y` as long as they are not both passed to `sort`.

Next come the range-specific requirements: `[first, last)` shall be a valid range upon calling the function. The caller shall also make sure that the range doesn't change while the algorithm is executed, for instance from another thread.

Finally, there are sort-specific guarantees offered by the function:

- the resulting range is sorted with respect to `comp` and `proj`,
- the output range is a permutation of the input range,
- the number of calls to `comp` and `proj` is  $\mathcal{O}(N \log N)$ , where  $N$  is `last - first`.

## 2. Encoding in contract assertions

### 2.1 Checking concept modeling?

Checking if a given type models a concept is not possible in general. As noted above, there is no Boolean predicate that would check if the value of the new object is the same as the original value of the moved-from object. Especially that the tested type may not provide a copy constructor or equality comparison. Also, even reflecting it in the interface of `sort` appears wrong. In order for the contract assertion to be useful, it should reflect the part of the contract that the author has reasons to believe could be violated.

It is more likely that a user will provide a predicate that does not model `strict_weak_order`. This one can be evaluated, although with a quadratic complexity:

```

template<random_access_iterator I, sentinel_for<I> S, class Comp, class Proj>
    requires sortable<I, Comp, Proj>
constexpr bool
    is_strict_weak_order(I first, S last, Comp comp = {}, Proj proj = {})
{
    for (I i = first; i != last; ++i)
        for (I j = first; j != last; ++j)
            if (not test_strict_weak_order(comp, proj(i), proj(j)))
                return false;
    return true;
}

```

```

template<random_access_iterator I, sentinel_for<I> S, class Comp, class Proj>
    requires sortable<I, Comp, Proj>
constexpr I ranges::sort(I first, S last, Comp comp = {}, Proj proj = {})
    pre(is_strict_weak_order(first, last, comp, proj));

```

Calling a quadratic algorithm of an otherwise  $O(N \log N)$  algorithm would break the complexity guarantees. We could write it down, in the specification of the Standard Library, but now it could stir a concern. Will it be evaluated?

We could make a general clause in the Standard Library that an implementation is allowed not to put contract annotations on functions even if they are declared in the synopsis.

Alternatively, we could introduce a modification to the contract assertion syntax which says that these contract assertions even if declared are not runtime-evaluated. This is what [\[P0542R5\]](#) calls an **audit** level. A special mode is required to trigger the evaluation of such predicates.

```

template<random_access_iterator I, sentinel_for<I> S, class Comp, class Proj>
    requires sortable<I, Comp, Proj>
constexpr I ranges::sort(I first, S last, Comp comp = {}, Proj proj = {})
    pre audit(is_strict_weak_order(first, last, comp, proj));

```

We cannot exhaustively test if a predicate is *equality-preserving*. We could call the predicate three times for each argument pair to see if it gives the same response all three times, and if the values of the arguments didn't change much. However, we cannot use `operator==`, as it is never required by the concept constraints. We could say, "provided that `operator==` is available, perform additional tests". This could read like this:

```

template<random_access_iterator I, sentinel_for<I> S, class Comp, class Proj>
    requires sortable<I, Comp, Proj>
constexpr I ranges::sort(I first, S last, Comp comp = {}, Proj proj = {})
    pre (maybe_is_equality_preserving(first, last, comp, proj))
    requires equality_comparable<typename iterator_traits<I>value_type>;

```

But this is still only a heuristic. Checking the result thrice does not mean that it *always* returns the same value for the same arguments.

## 2.2 Checking range guarantees?

The above predicates that we invented (`is_strict_weak_order`, `maybe_is_equality_preserving`) are still imperfect, as they have a precondition themselves: that `[first, last)` is a valid range. This cannot be checked with a C++ predicate. But it can be described formally. We can say that `[first, last)` is a valid range when the following algorithm finishes and doesn't have undefined behavior.

```

template<random_access_iterator I, sentinel_for<I> S>
void valid_range_axiom(I first, S last)
{
    for (; first != last; ++first) {}
}

```

Declaring contract assertions with inexpressible predicates still has value. First, users and IDEs can read them. Second, they can be used as a basis for control flow analysis, symbolic analysis and value range analysis.

To support this type of a predicate, we would have to introduce a new kind of a declaration: either of a contract assertion or of a function:

```

template<random_access_iterator I, sentinel_for<I> S>
    axiom is_valid_range(I first, S last);    // declared but never defined

template<random_access_iterator I, sentinel_for<I> S, class Comp, class Proj>
    requires sortable<I, Comp, Proj>

```

```
constexpr I ranges::sort(I first, S last, Comp comp = {}, Proj proj = {})
    pre axiom(is_valid_range(first, last)); // modify the meaning of a precondition
```

Or alternatively:

```
template<random_access_iterator I, sentinel_for<I> S>
    axiom is_valid_range(I first, S last); // modify the meaning of a function declaration

template<random_access_iterator I, sentinel_for<I> S, class Comp, class Proj>
    requires sortable<I, Comp, Proj>
constexpr I ranges::sort(I first, S last, Comp comp = {}, Proj proj = {})
    pre (is_valid_range(first, last));
```

[\[P0542R5\]](#) proposed **axiom**-level assertions. [\[P2176R0\]](#) discusses the latter approach in more detail.

## 2.3 Storing a postcondition state

Finally, we would need to express the postcondition that the resulting sequence is a permutation of the initial sequence. In order to do that, we would have to observe both the initial and the result sequence at the same time. This means that we would have to store a copy of the input sequence and keep it until a postcondition is evaluated or an exception is thrown. We could introduce a lambda-capture-like syntax for making copies of the selected program state:

```
template<random_access_iterator I, sentinel_for<I> S, class Comp, class Proj>
    requires sortable<I, Comp, Proj>
constexpr I ranges::sort(const I first, const S last, Comp comp = {}, Proj proj = {})
    post audit [in = vector(first, last)] (is_permutation(first, last, in.begin(), in.end()));
```

So, in order to express this postcondition, we would have to capture the state of the entire range. Here we chose to store it in a **vector**. The choice is absolutely arbitrary. Note that while the algorithm **sort** is container-agnostic, now the postcondition ties the declaration of **sort** to a specific container. The predicate in the postcondition assertion expresses more than the plain language contract. While the latter talks about the state of ranges before and after, the former now talks about creating a **vector**.

With creation of a vector, we no longer have a nice, pure predicate. It now can allocate memory, perform an  $\mathcal{O}(N)$  copying and potentially throw. This postcondition is not only a "predicate". It is also an action executed when the function starts. It is difficult to imagine that this type of postcondition would be used for static analysis. We also need to mention that this check has complexity of  $\mathcal{O}(N^2)$ , which is greater than the complexity of the sort.

## 2.4 Putting all this together

If we combine all the above assertions together, we would get:

```
template<random_access_iterator I, sentinel_for<I> S, class Comp, class Proj>
    requires sortable<I, Comp, Proj>
constexpr I ranges::sort(const I first, const S last, Comp comp = {}, Proj proj = {})
    pre axiom (is_valid_range(first, last))
    pre audit (is_strict_weak_order(first, last, comp, proj))
    pre audit (maybe_is_equality_preserving(first, last, comp, proj))
    requires equality_comparable<typename iterator_traits<I>value_type>
    post (ranges::is_sorted(first, last, comp, proj))
    post audit [in = vector(first, last)] (is_permutation(first, last, in.begin(), in.end()));
```

That is admittedly a wall of text. Interestingly, only one of these assertions can be expressed with what is proposed in [\[P2900R7\]](#).

When adding contract assertions to the Standard Library components, one has to ask the question: what is the goal? There could be at least two answers:

1. To give *every* possible hint to the tools to help them detect contract violations.
2. To document the parts of the interface that have not already been documented through types, concepts and specifiers.

The above declaration satisfies the first goal. If we aim for the second goal, we could put fewer assertions. Concept **sortable** already implies the semantic properties of a strict weak order and an equality-preserving function. So we could skip the corresponding assertions. Also the requirement that `[first, last)` is a valid range is so obvious that we could think of skipping it. Also, a safer alternative is to use the range version of the algorithm, where we have a single object that clearly represents a range. But if we only got rid of the contracts that represent the semantic requirements of contracts, we would end up with a slightly slimmer declaration.

```
template<random_access_iterator I, sentinel_for<I> S, class Comp, class Proj>
    requires sortable<I, Comp, Proj>
```

```
constexpr I ranges::sort(const I first, const S last, Comp comp = {}, Proj proj = {})  
    pre axiom (is_valid_range(first, last))  
    post (ranges::is_sorted(first, last, comp, proj))  
    post audit [in = vector(first, last)] (is_permutation(first, last, in.begin(), in.end()));
```

### 3. References

- [P0542R5] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++"  
(<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0542r5.html>).
- [P2176R0] — Andrzej Krzemiński, "A different take on inexpressible conditions"  
(<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2176r0.html>).
- [P2345R0] — Sean Parent, "Relaxing Requirements of Moved-From Objects",  
(<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2345r0.pdf>).
- [P2900R7] — Joshua Berne, Timur Doumler, Andrzej Krzemiński, "Contracts for C++",  
(<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2900r7.pdf>).